

CelestialMechanics.Desktop — Native WPF Simulation IDE

Project: Celestial Mechanics Desktop Simulator

Framework: WPF (.NET 8) + OpenGL (Silk.NET)

Architecture: MVVM (CommunityToolkit.Mvvm)

Date: April 6, 2026

Author: Auto-Generated Analysis

Table of Contents

- 1. Executive Summary & Key Metrics
- 2. Application Architecture Overview
- 3. UI Layer Structure & File Inventory
- 4. Navigation & Modal System
- 5. IDE Workspace Layout
- 6. Theming & Design System
- 7. MVVM Implementation Analysis
- 8. 3D Viewport & Rendering Pipeline
- 9. Shader-Based Procedural Rendering
- 10. User Interaction Model
- 11. Panel Components Deep-Dive
- 12. Services Layer Analysis
- 13. ImGui Debug Overlay (App Project)
- 14. Strengths & Achievements
- 15. Gaps & Areas for Improvement
- 16. Recommendations & Roadmap

1. Executive Summary & Key Metrics

The **CelestialMechanics.Desktop** project is a native Windows desktop application built with WPF and .NET 8, serving as the primary GUI front-end for an N-body gravitational simulation engine. The application features a cinematic dark theme, a multi-stage modal navigation system, a full 3-panel IDE workspace, and a hardware-accelerated OpenGL 3D viewport with procedural shader-based rendering of celestial bodies.

48

Total UI/Frontend Files

11

ViewModels

14

GLSL Shaders

5

Modal Screens

4

IDE Panels

10

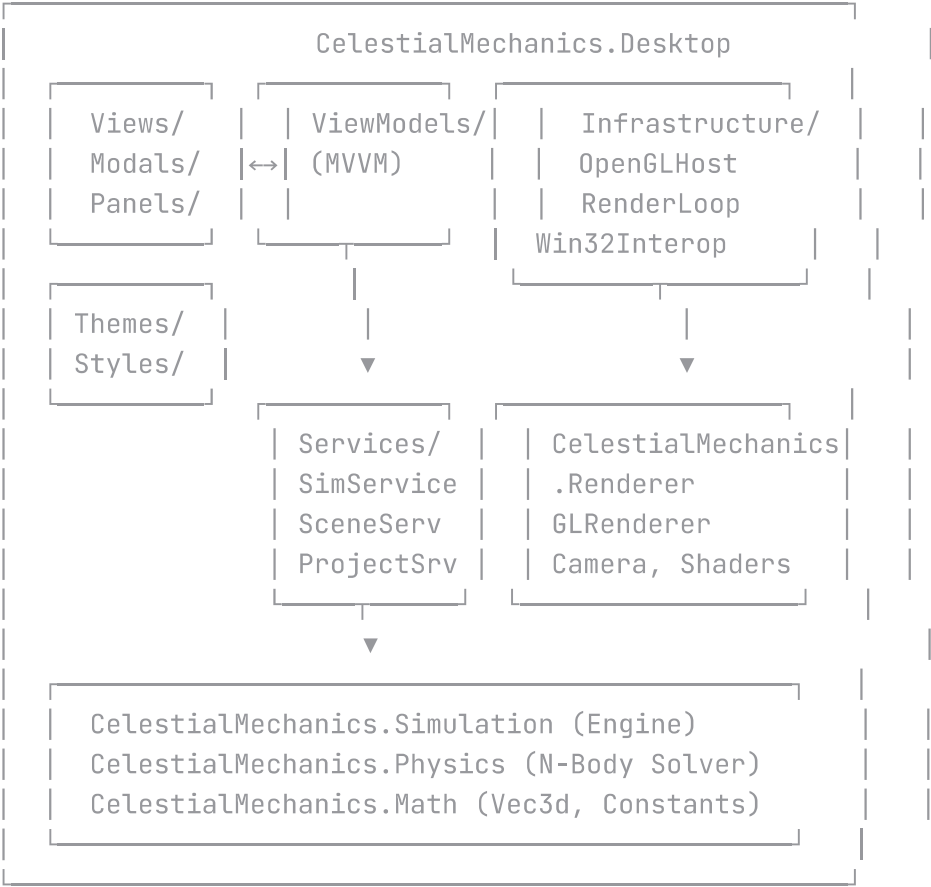
Body Types Rendered

Technology Stack: WPF (.NET 8) · Silk.NET OpenGL · CommunityToolkit.Mvvm 8.4 · Win32 Interop (WGL/HwndHost) · GLSL 330 Core · JSON Serialization

2. Application Architecture Overview

The frontend spans two primary projects within the solution, plus shader assets from the Renderer project:

Project	Role	Key Technologies
CelestialMechanics.Desktop	WPF application — Views, ViewModels, themes, services, infrastructure	WPF, MVVM, Win32 Interop
CelestialMechanics.Renderer	OpenGL rendering engine — camera, shaders, instanced rendering	Silk.NET OpenGL, GLSL 330
CelestialMechanics.App	Standalone app with ImGui debug overlay (developer tool)	ImGui.NET, Silk.NET



3. UI Layer Structure & File Inventory

3.1 Desktop Project File Map

Directory	Files	Purpose
Views/	MainWindow.xaml/.cs, ViewportPanel.xaml/.cs	Primary window and OpenGL viewport host
Views/Modals/	5 modals × (XAML + .cs) = 10 files	ModeSelection, SimulationMenu, NewProject, ProjectsList, FileMenu
Views/Panels/	4 panels × (XAML + .cs) = 8 files	SceneOutliner, BodyInspector, SimulationSettings, BodyTypePalette
ViewModels/	11 C# files	MainWindowVM, BodyInspectorVM, SceneOutlinerVM, SimSettingsVM, etc.
Themes/	Colors.xaml, SpaceDarkTheme.xaml	Color tokens and base control templates
Styles/	Theme.xaml	Extended cinematic styles, modal styles, inspector styles
Converters/	BoolToVisibilityConverter.cs	XAML data binding converters
Services/	SimulationService, SceneService, ProjectService	Thread-safe engine wrappers
Infrastructure/	OpenGLHost, RenderLoop, Win32Interop	Native HWND, WGL context, render thread
Models/	ProjectInfo, SceneNodeItem, SimulationSaveState	Data transfer objects

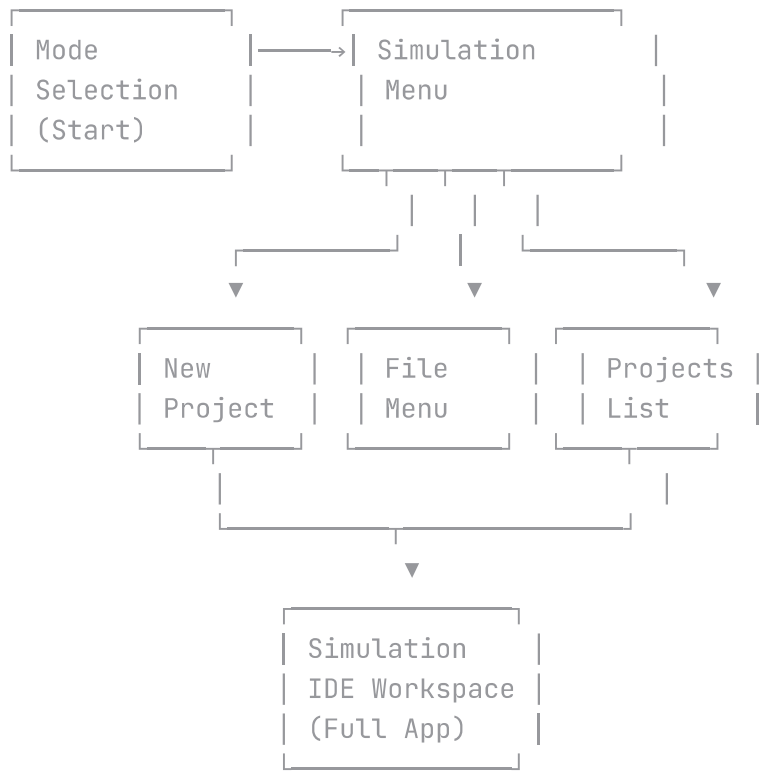
3.2 Renderer Project (Frontend-Relevant)

File	Lines	Purpose
GLRenderer.cs	217	Multi-pass rendering orchestrator (5 render passes)
Camera.cs	237	Orbit camera with smooth interpolation and fly-to animation
InstancedSphereRenderer.cs	~220	GPU-instanced sphere rendering for celestial bodies
TrailRenderer.cs	~190	Orbital trail GL_LINE_STRIP rendering with ring buffer
StarfieldRenderer.cs	~120	Background star particle system
GridRenderer.cs	~75	Reference grid on XZ plane
SelectionHelper.cs	~80	CPU-side ray-sphere intersection for body picking
Shaders/ (14 files)	~1,500 total	GLSL vertex/fragment shaders for all visual elements

4. Navigation & Modal System

The application implements a **multi-stage modal navigation flow** before entering the main IDE. This is managed through a `NavigationState` enum and DataTriggers in XAML that control modal visibility.

4.1 Navigation Flow



4.2 Navigation States

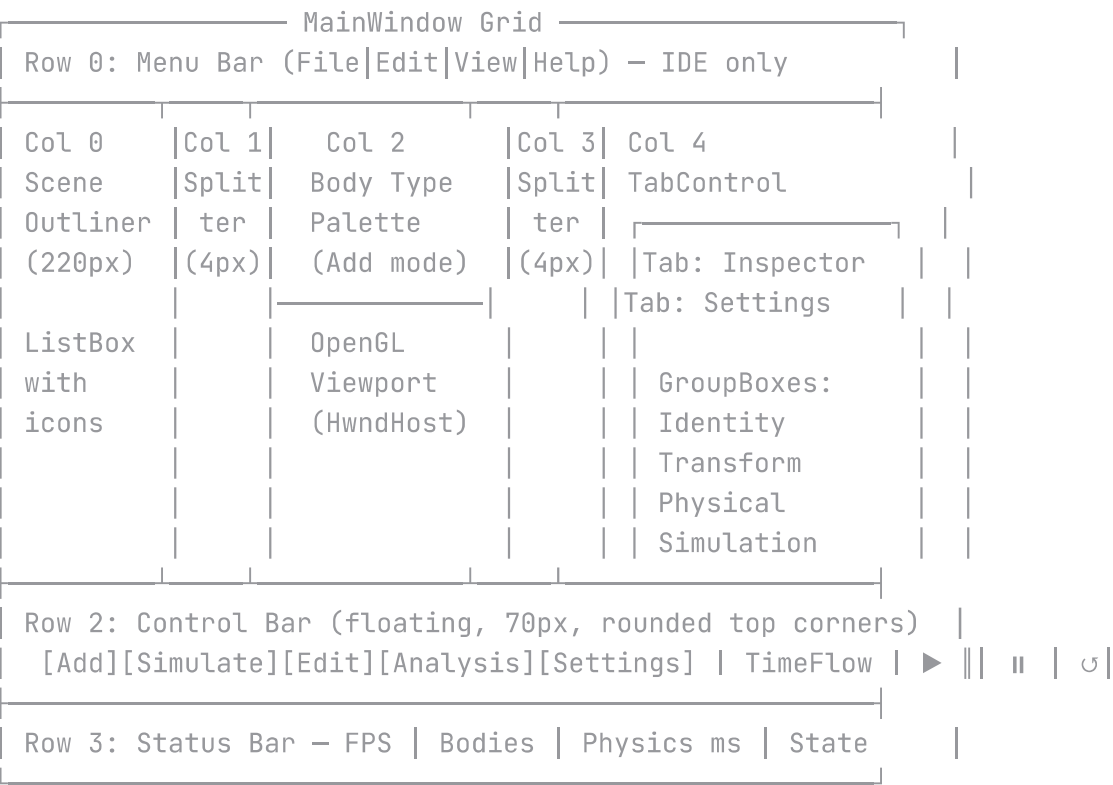
State	Modal	Description
ModeSelection	ModeSelectionModal	Landing screen — choose Simulation or Observation (disabled)
SimulationMenu	SimulationMenuModal	New Project, File, Projects, Settings options
NewProject	NewProjectModal	Name and location form for creating a new simulation
ProjectsList	ProjectsListModal	Browse and open previously saved projects
FileMenu	FileMenuModal	File operations submenu
SimulationIDE	None (IDE active)	Full 3-panel workspace with viewport

All modals share consistent visual design: 400×350px centered panels with #161B22 background, 16px corner radius, drop shadow, and the same button style hierarchy (Primary → Secondary → Back).

Background Video: The modal screens feature a `MediaElement` playing a looping background simulation video (`background_simulation.mp4`) with a 20px blur and 40% opacity, creating an immersive cinematic feel. If the video is unavailable, it falls back gracefully to the dark background.

5. IDE Workspace Layout

Once a project is opened, the UI transitions to the full Simulation IDE — a 4-row, 5-column grid layout:



5.1 Panel Dimensions & Behavior

Panel	Default Width	Min	Max	Toggleable
Scene Outliner (left)	220px	150px	400px	Yes (View menu)
Viewport (center)	Star (*)	400px	∞	Always visible
Inspector/Settings (right)	260px	200px	500px	Yes (View menu)

Panels are separated by interactive `GridSplitter` controls (4px wide) that highlight cyan on hover. All panels can be toggled via the View menu (`ShowSceneOutliner` , `ShowInspector`).

6. Theming & Design System

6.1 Color Palette

The design system is built on a carefully curated "Space Dark" palette defined in `Colors.xaml` (19 colors) and extended in `Theme.xaml` (6 additional cinematic colors):

Category	Token	Hex	Usage
Background	BackgroundPrimary	 #0A0E17	Window background, deepest layer
Background	BackgroundSecondary	 #121829	Side panels, status bar
Background	BackgroundTertiary	 #1A2236	Group box headers, text inputs
Surface	Surface	 #1E2740	Buttons, cards, selected items
Surface	SurfaceHover	 #243050	Hover states on all interactive elements
Accent	AccentCyan	 #4FC3F7	Primary accent — selected tabs, active states, borders
Accent	AccentPurple	 #7C4DFF	Secondary accent — text selection highlight
Accent	GlowCyan	 #00E5FF	Pressed state glow effect
Text	TextPrimary	 #E8EAF0	Primary content text
Text	TextSecondary	 #8892B0	Labels, panel headers, secondary info
Status	StatusGreen/Yellow/Red		Simulation state indicators

6.2 Styled Control Templates (27 styles)

The theme system provides complete custom `ControlTemplate` overrides for all WPF controls. Key styles include:

Style Key	Target Control	Features
ModeButtonStyle	Button	8px corner radius, cyan DropShadow glow on hover, active state via DataTrigger
GlowToggleButtonStyle	Button	Cyan border, 12px blur glow on hover, semi-transparent cyan fill on press
ModalNavButtonStyle	Button	Left-border accent indicator, 8px radius, full-width menu items
ModalPrimaryButtonStyle	Button	Solid cyan background, dark text, opacity reduction on hover
SpaceTabControlStyle	TabControl	Borderless tabs with 2px bottom accent on selected tab
SpaceTreeViewItemStyle	TreeViewItem	Custom expander arrows (cyan when open), left-border selection indicator
InspectorGroupBoxStyle	GroupBox	Surface header bar, tertiary content background
SpaceSliderStyle	Slider	3px track, 14px ellipse thumb, cyan filled portion
FormTextBoxStyle	TextBox	6px corner radius, cyan border on focus, purple selection
CompactStatusBarStyle	StatusBar	24px height, 11pt font, menu-bar-background color

6.3 Typography

- **Primary font:** Segoe UI — used for all UI text (13pt default)
- **Monospace font:** Consolas — used for numeric displays, body IDs, time values
- **Font sizes:** 10pt (panel headers, labels), 11pt (inspector fields, status), 12pt (mode buttons), 13pt (general text), 14pt (modal buttons), 18pt (modal headers), 22pt (cover title)

7. MVVM Implementation Analysis

7.1 ViewModel Architecture

The application uses **CommunityToolkit.Mvvm 8.4** with source generators for all ViewModels. The `MainWindowViewModel` serves as the composition root, owning all services and child ViewModels.

ViewModel	Lines	Responsibilities
<code>MainWindowViewModel</code>	707	Navigation state machine, mode switching, service orchestration, simulation commands, UI timer (20Hz), body placement, save/load, 30+ observable properties
<code>SceneOutlinerViewModel</code>	~130	ObservableCollection of SceneNodeItems, selection syncing, delete commands
<code>BodyInspectorViewModel</code>	~130	Two-way bound body properties (position, velocity, mass, radius), apply changes command
<code>SimulationSettingsViewModel</code>	~160	All PhysicsConfig properties (integrator, Barnes-Hut, collisions, relativistic), apply/close commands
<code>ModeSelectionViewModel</code>	~25	SelectSimulation and Exit event delegates
<code>SimulationMenuViewModel</code>	~30	NewProject, File, Projects, Back navigation events
<code>NewProjectViewModel</code>	~75	Project name/location form, browse dialog, project creation
<code>ProjectsListViewModel</code>	~50	ObservableCollection of projects, refresh, open selection
<code>FileMenuViewModel</code>	~30	Back navigation only (stub for future file operations)
<code>ModeState</code>	~25	UiMode enum: Idle, AddPlacement, Edit, Simulate, Analyse
<code>NavigationState</code>	~20	Enum for modal navigation flow (6 states)

7.2 Data Binding Patterns

- **[ObservableProperty]** — Source-generated properties with automatic `PropertyChanged` notifications (used extensively — 30+ properties in `MainWindowViewModel` alone)
- **[RelayCommand]** — Source-generated `ICommand` implementations for toolbar, menu, and modal actions
- **[NotifyPropertyChangedFor]** — Cascading property change notifications (e.g., `NavState` change notifies `IsModalVisible`, `IsIdleActive`, `IsStatusBarVisible`, `IsControlBarVisible`)
- **DataTrigger binding** — Modal visibility controlled via `DataTriggers` bound to `NavState` enum values
- **Event-based wiring** — Child ViewModels communicate via C# events (e.g., `ProjectCreated` , `BodySelected` , `DeleteRequested`)

8. 3D Viewport & Rendering Pipeline

8.1 OpenGL Integration Architecture

The 3D viewport is implemented through a sophisticated multi-layer architecture:

1. **OpenGLHost (HwndHost)** — Creates a native child `HWND` window with `CS_OWNDNC` class style, sets up a WGL OpenGL context, and provides a `Silk.NET.OpenGL.GL` API object. DPI-aware pixel calculation via `VisualTreeHelper.GetDpi()` .
2. **RenderLoop** — Runs on a dedicated render thread at 60 FPS, owns the WGL context via `wglMakeCurrent` , handles frame pacing and FPS tracking.
3. **GLRenderer** — Orchestrates the 5-pass rendering pipeline, managing all sub-renderers and shader programs.
4. **ViewportPanel** — WPF `UserControl` wrapping the OpenGL host, handling all mouse/keyboard input delegation to the camera and ViewModel.

8.2 Rendering Passes (Per Frame)

Pass	Renderer	Shader	Description
1	StarfieldRenderer	starfield.vert/frag	Background star particles (depth test disabled)
2	GridRenderer	grid.vert/frag	XZ-plane reference grid (alpha blended)
3	TrailRenderer	trail.vert/frag	Orbital trajectory trails (GL_LINE_STRIP)
4	InstancedSphereRenderer	sphere.vert/frag	Celestial bodies with procedural texturing (~460-line fragment shader)
5	LineRenderer	line.vert/frag	Velocity arrow vectors (optional)

8.3 Camera System

The `Camera` class implements an orbit camera with smooth interpolation:

- **Orbit:** Yaw/Pitch rotation around an adjustable target point
- **Pan:** Shift+drag moves the target in the camera's right/up plane
- **Zoom:** Scroll wheel adjusts orbital distance (0.1–10,000 range)
- **FOV Zoom:** Shift+Scroll adjusts field of view (10°–120°)
- **WASD+QE:** Keyboard navigation at 60Hz polling rate
- **Fly-to:** Smooth cubic ease-in-out animation (0.8s duration) to focus on a body
- **Damping:** All parameters use exponential smoothing (factor 8.0) for butter-smooth motion

9. Shader-Based Procedural Rendering

The `sphere.frag` shader (459 lines) is the visual centerpiece of the application. It implements procedural texturing for 10 distinct body types using Simplex noise (fBm), avoiding any texture files entirely.

Body Type	Visual Effect	Technique
Star	Animated plasma, sunspots, convection cells, limb darkening	5-octave fBm + time animation
Planet	Earth-like continents, oceans, polar ice caps	6-octave fBm with altitude-based coloring
Gas Giant	Jupiter-like bands, Great Red Spot, swirling storms	Latitude bands + turbulence noise + spot mask
Rocky Planet	Mars-like terrain, canyons, crater rims, polar frost	5-octave fBm + crater noise
Moon	Craters (large + small), maria, gray highlands	Multi-scale noise for different crater sizes
Asteroid	Rough irregular rock, regolith patches	4-octave fBm + high-frequency roughness
Neutron Star	Blue-white pulsing, magnetic pole beams, surface crackle	Sinusoidal pulse + high-freq animated noise
Black Hole	Dark center, Fresnel rim glow, photon ring, accretion swirl	Fresnel ³ + ring masking + animated noise
Comet	Icy/dusty surface, outgassing bright spots	fBm + spot threshold noise
Custom	Solid instance color	Pass-through from vertex attribute

Lighting model: Blinn-Phong (ambient + diffuse + specular) with per-body-type shininess, Fresnel rim light, atmospheric glow for planets, and animated cyan selection highlight (pulsing at 4Hz).

10. User Interaction Model

10.1 UI Modes (State Machine)

Mode	Left Click	Right Click	Cursor	Palette
Idle	Select body (raycast)	Deselect	Arrow	Hidden
AddPlacement	Place body at XZ plane intersection	Cancel placement	Cross	Visible (10 body types)
Edit	Select body (raycast)	Deselect	Hand	Hidden
Simulate	Orbit camera (drag)	Deselect	Arrow	Hidden
Analyse	Select body (raycast)	Deselect	Arrow	Hidden

10.2 Keyboard Shortcuts

Key	Action	Scope
Ctrl+S	Save project	Global (InputBinding)
Ctrl+O	Open project	Global (InputBinding)
Delete	Delete selected body	Global (InputBinding)
W/A/S/D	Camera navigation (forward/left/back/right)	Viewport focused
Q/E	Camera up/down	Viewport focused
Home	Reset camera to default	Viewport focused
F	Focus camera on selected body	Viewport focused
Double-click	Fly camera to clicked body	Viewport

11. Panel Components Deep-Dive

11.1 Scene Outliner Panel

Flat `ListBox` displaying all simulation bodies with icon glyphs, name, and type label. Features cyan left-border selection indicator, hover highlighting, and right-click context menu with Delete option. Selection syncs bidirectionally with the `BodyInspectorViewModel` and viewport selection highlight.

11.2 Body Inspector Panel

Scrollable panel with 4 `GroupBox` sections:

- **Identity** — Editable Name (`TextBox`), Type (`ComboBox`), read-only ID
- **Transform** — Position X/Y/Z and Velocity X/Y/Z (6 editable `TextBoxes`, F6 format, `UpdateSourceTrigger=LostFocus`)
- **Physical** — Mass, Radius (editable, G6 format), Density (read-only)
- **Simulation** — Active and Collidable checkboxes

Shows "Select a body to inspect" placeholder when nothing is selected. Includes an "Apply Changes" button that writes values back to the physics engine.

11.3 Simulation Settings Panel

7 `GroupBox` sections exposing the entire `PhysicsConfig` surface:

- **Integrator** — `ComboBox` (Euler/Verlet/RK4)
- **Performance** — Deterministic, Parallel, SIMD, SoA checkboxes
- **Algorithm** — Barnes-Hut toggle + Theta slider (0.0–1.0, conditionally enabled)
- **Physics** — Collisions, Adaptive Timestep toggles
- **Timestep** — dt, Min dt, Max dt numeric inputs
- **Relativistic** — Post-Newtonian, Lensing, Accretion Disks, Gravitational Waves, Jets
- **Softening** — Epsilon value + Mode dropdown (Constant/Plummer)

11.4 Body Type Palette

Horizontal strip (44px height) visible only in Add mode, containing 10 body type buttons with Unicode glyph icons: ★ Star, ● Planet, ☉ Gas Giant, ▣ Rocky, ◉ Moon, ◇ Asteroid, ◆ Neutron, ○ Black Hole, ☄ Comet, ♦ Custom.

12. Services Layer Analysis

Service	Lines	Thread Safety	Key Features
<code>SimulationService</code>	~140	Dedicated sim thread (~1kHz), <code>Interlocked</code> for metrics, <code>WithEngineLock</code> callback	Play/Pause/Step/Reset, body CRUD, integrator switching, time scale
<code>SceneService</code>	~90	UI thread only	Maintains <code>SceneGraph</code> ↔ <code>SimulationEngine</code> body mapping, selection management
<code>ProjectService</code>	~90	UI thread only	Project directory creation, project discovery, metadata persistence

13. ImGui Debug Overlay (App Project)

The `CelestialMechanics.App` project provides a separate standalone application with an ImGui-based overlay used for development and debugging. It includes 6 dockable panels:

- **Simulation Controls** — Play/Pause/Step/Reset with color-coded state indicator
- **Energy Monitor** — Kinetic, Potential, Total energy with scrolling graph (200-point history), drift percentage with green/yellow/red color coding
- **Performance** — FPS graph (120-point history), physics/render timing, frame budget progress bar
- **Integrator Selector** — Combo dropdown with description text for each method
- **Add Body** — Template selector with auto-populated mass/radius, position/velocity inputs
- **Body Inspector** — ListBox selector, position/velocity/acceleration readout with X/Y/Z color coding

Note: The ImGui overlay is a separate application from the WPF Desktop app. The Desktop project has replicated most of this functionality in its native WPF panels, though the Energy Monitor graph and FPS graph are not yet replicated in the WPF version.

14. Strengths & Achievements

1. **Professional-grade theming:** 27 custom control templates with consistent color tokens, hover/press/disabled/selected states across all controls. The design language is cohesive and visually impressive.
2. **Sophisticated procedural rendering:** 460-line fragment shader with Simplex noise fBm generates unique, animated visuals for 10 body types — no textures needed. Star convection, gas giant bands, and black hole rim glow are exceptionally well-implemented.
3. **Proper threading architecture:** Three independent threads (UI at 20Hz, Render at 60Hz, Physics at ~1kHz) with thread-safe communication via locks and **Interlocked** atomic operations.
4. **Smooth camera system:** Exponential damping on all 5 camera parameters (yaw, pitch, distance, target, FOV) with fly-to animation using cubic ease-in-out interpolation.
5. **Complete MVVM implementation:** 11 ViewModels with source-generated properties/commands, event-driven inter-VM communication, and minimal code-behind (only View-specific concerns like OpenGL init and video playback).
6. **Full simulation settings exposure:** All 20+ PhysicsConfig parameters accessible through the Settings panel, including advanced features like Barnes-Hut theta, relativistic effects, and adaptive timestep.
7. **Cinematic modal navigation:** Multi-stage entry flow with background video, blur effects, and consistent modal styling creates a polished first impression.
8. **Body placement system:** Complete Add mode workflow with palette selection, cursor change, XZ plane raycasting for placement, and right-click cancellation.

15. Gaps & Areas for Improvement

Missing Features (from README.md roadmap):

Feature	Status	Difficulty
Snapshot Timeline / Time Scrubber	Not started	Medium — SnapshotManager API exists
Event Log Panel	Not started	Low — EventBus API exists
Real-time Energy Chart	Exists in ImGui, not in WPF	Medium — needs canvas or charting lib
Undo/Redo System	Not started	Medium — can leverage snapshots
GPU Engine Mode Selector	Not started	Low — dropdown wiring
Dependency Injection Container	Not implemented	Low — services are manually constructed
Scene Tree Hierarchy (TreeView)	Flat ListBox instead	Medium — SceneGraph API supports hierarchy
Body Creation Dialog (detailed)	Inline palette only	Medium — custom mass/velocity inputs needed
Additional Converters	Only BoolToVisibility exists	Low — Vec3dToString, BodyTypeToIcon, etc.
Accretion Disk / Lensing Renderers	Renderer classes exist but not integrated into pipeline	Medium — separate render passes needed

15.1 Technical Observations

- **MainWindowViewModel is overly large** (707 lines) — handles navigation, modes, placement, simulation control, save/load, and metric updates. Should be decomposed into focused ViewModels.
- **No input validation** on inspector TextBoxes — users can enter invalid values for mass, radius, etc. without warning.
- **Body Type Palette commands** use a workaround with `Tag` property and `Int32.MinValue` CommandParameter — needs proper enum-based command parameter binding.
- **No error handling UX** — file load errors are silently caught, save failures are invisible to users.
- **Accessibility** — no ARIA-equivalent automation properties, no focus indicators on custom-templated controls, no high-contrast theme.

- **No animations** between navigation states — modals appear/disappear instantly without transitions.

16. Recommendations & Roadmap

Phase 1 — Quick Wins (1-2 weeks)

- Add modal transition animations (fade/slide using WPF Storyboards)
- Implement input validation on inspector fields with visual error indicators
- Add missing converters: `Vec3dToStringConverter` , `EngineStateToColorConverter` , `BodyTypeToIconConverter`
- Replace Scene Outliner flat ListBox with hierarchical TreeView using existing styles
- Add error notification system (snackbar/toast style) for save/load failures

Phase 2 — Feature Completion (2-4 weeks)

- Implement Snapshot Timeline Panel with horizontal slider at viewport bottom
- Port the ImGui Energy Monitor graph to a WPF canvas-based rolling chart
- Add Event Log panel with ListView, color-coded by event type
- Create detailed Add Body dialog with custom mass/position/velocity inputs
- Integrate AccretionDiskRenderer and RelativisticRenderer into the render pipeline

Phase 3 — Architecture Improvements (2-4 weeks)

- Decompose MainWindowViewModel into NavigationViewModel, SimulationControlViewModel, PlacementViewModel
- Add DI container (Microsoft.Extensions.DependencyInjection) for service registration
- Implement undo/redo using the SnapshotManager
- Add UI virtualization for Scene Outliner when body count exceeds 10,000
- Add automated UI tests using WPF automation framework

Phase 4 — Polish (Ongoing)

- Add keyboard shortcut overlay (Space = play/pause, R = reset, etc.)
- Implement draggable/dockable panel system
- Add theme switching (light mode optional)
- Accessibility improvements (automation properties, keyboard focus management)

- Performance profiling and optimization for large body counts

Celestial Mechanics Desktop — UI & Frontend Analysis Report

Generated April 6, 2026 · 48 files analyzed across 3 projects